Transformers

Roteiro do vídeo-apresentação
Inteligência Artificial · Engenharia de Computação

> A representação de um token depende do contexto: um Transformer calcula, em paralelo, quanto cada posição deve aproveitar de todas as outras, e aprende esses pesos otimizando a previsão de tokens.

Apresentador	Papel	Tempo
Palos	Contexto e representação	08:45
Davi	Mecanismo de atenção	08:45
Mendes	Arquitetura e treinamento	08:50
Oliveira	Inferência e aplicações	08:50

Duração total: 35:10 · 44 cenas · 13 capítulos

Visão geral

A metáfora: a planta cotada

Em desenho técnico, uma peça só fica definida por suas **cotas**: as linhas que dizem como cada elemento se relaciona com os demais. Sem cotas, há um contorno sem medida. O vídeo trata o texto da mesma forma:

- cada **token** é uma peça sobre a prancha;
- cada **peso de atenção** é uma linha de cota entre duas peças, com espessura proporcional ao peso;
- cada **capítulo** é uma prancha de detalhe, numerada de 01 a 13 no carimbo;
- a **prancha final** é o desenho de conjunto, no qual os detalhes se encaixam no diagrama completo do modelo.

Onde a metáfora para de valer

O roteiro diz em voz alta onde a metáfora para de valer (cena C04-S01), para que ela não vire erro conceitual:

- uma cota real é medida e exata; um peso de atenção é aprendido, contínuo e não garante uma relação verdadeira;
- a peça do desenho não muda quando é cotada; aqui, o vetor do token é de fato recalculado a cada camada;
- espessura de linha não é explicação: pesos altos não provam por que o modelo decidiu algo (Jain; Wallace, 2019).

Arco narrativo

O vídeo é uma sequência de problemas, em que cada solução cria a pergunta seguinte:

1. **Palos:** RNNs leem em série e esquecem. A atenção de Bahdanau ajuda, mas ainda se apoia na recorrência. Pergunta: *e se a atenção fosse tudo?*
2. **Davi:** a self-attention responde com uma conta simples, verificável à mão. Pergunta: *se cada token olha para todos ao mesmo tempo, onde foi parar a ordem?*
3. **Mendes:** posição, bloco, máscara e treino constroem a máquina. Pergunta: *como ela gera texto depois de treinada?*
4. **Oliveira:** inferência, custo, aplicações e limites. Fecha com o desenho de conjunto e volta à frase-tese.

Mapa de tempo

Cap.	Título	Apresentador	Início	Fim
C01	Abertura e integrantes	Palos	00:00	01:25
C02	Por que não bastavam as RNNs	Palos	01:25	05:00
C03	Tokenização e embeddings	Palos	05:00	08:45
C04	Self-attention	Davi	08:45	15:45
C05	Multi-head attention	Davi	15:45	17:30
C06	Codificação posicional	Mendes	17:30	19:55
C07	O bloco completo	Mendes	19:55	23:45
C08	Treinamento	Mendes	23:45	26:20
C09	Inferência	Oliveira	26:20	29:35
C10	Custo e variantes eficientes	Oliveira	29:35	30:40
C11	Famílias e aplicações	Oliveira	30:40	32:45
C12	Limitações e questões em aberto	Oliveira	32:45	33:55
C13	Encerramento	Oliveira	33:55	35:10

Como usar a animação

Na pasta do projeto, rode `python -m http.server 8000` e abra `http://localhost:8000/app/`. Na tela inicial, escolha **Apresentar**, **Ensaiar** ou **Gravar**; este último deixa a tela limpa, sem controles. Mexa o mouse para ver a barra, com linha do tempo clicável e índice de cenas.

Tecla	Ação
Clique, **Espaço**, →, **PageDown**	Próximo passo; no último, próxima cena. A tela inicial não entra no vídeo: comece a gravar e só então escolha o modo.
←, **PageUp**	Volta um passo.
M	Índice de cenas.
PageDown, dentro de uma demo	Sai da demo e avança. Setas, espaço e cliques pertencem à demo; a barra aparece com o mouse no rodapé.
D, dentro de uma demo	Troca a demo pela imagem de contingência, se algo der errado.
E	Mostra ou esconde tempo e texto da fala.
H	Ajuda com todos os atalhos.

Nas cenas a seguir, cada item de **Cliques** corresponde a um avanço do apresentador. O primeiro acontece ao entrar na cena. O texto falado é o que se diz, palavra por palavra; ▶ marca o momento de avançar, e o itálico entre parênteses é uma ação, que não se lê.

Palos

Contexto e representação · de 00:00 a 08:45

C01 · Abertura e integrantes

C01-S01 · Capa e frase-tese

00:00–00:45 · 00:45

Objetivo: apresentar o tema e a tese que guia os 35 minutos.

NA TELA

Folha vazia com a moldura. Título "Transformers". Subtítulo com disciplina e curso. À direita, uma frase de exemplo com três tokens ligados por linhas de cota de espessuras diferentes. Frase-tese.

CLIQUES

1. A moldura se desenha em sentido horário; o carimbo sobe 8 px.
2. Título e subtítulo entram.
3. As três cotas são desenhadas, da mais fina à mais grossa.
4. A frase-tese entra linha a linha.

> TEXTO FALADO
>
> *(Folha vazia. Esperar 3 s em silêncio.)*
>
> ▶ Olá. Este trabalho é da disciplina de Inteligência Artificial, do curso de Engenharia de Computação.
>
> ▶ O tema é o Transformer, a arquitetura por trás dos grandes modelos de linguagem atuais. Nos próximos trinta e cinco minutos, vamos construir esse modelo do zero, peça por peça, com código executado aqui na tela.
>
> ▶ A ideia que guia tudo cabe numa frase.
>
> ▶ A representação de uma palavra depende do contexto, e pesos aprendidos dizem quanto cada posição aproveita das outras. Cada capítulo é uma prancha técnica, como num projeto de engenharia, e a última reúne todas. Antes, quem somos nós e quem cuida de cada prancha.

Referências: Vaswani et al. (2017).

C01-S02 · Quem apresenta o quê

00:45–01:25 · 00:40

Objetivo: deixar claro quem são os integrantes e qual parte cada um apresenta.

NA TELA

Tabela de responsáveis em tela cheia, como lista de revisão de projeto. Colunas: nome e papel (lidos do config), capítulos, intervalo de tempo. Abaixo, uma linha do tempo horizontal de 35 min com os quatro blocos separados por marcas.

CLIQUES

1. Cada linha da tabela é escrita de cima para baixo, e o bloco correspondente aparece na linha do tempo.
2. Marcadores das 9 cenas com implementação surgem na linha do tempo.

> TEXTO FALADO
>
> Somos quatro. Eu sou Palos e abro com o contexto e a representação: por que as redes recorrentes não bastavam e como o texto vira números. Davi apresenta o mecanismo de atenção, o coração do modelo. Mendes monta a arquitetura completa e mostra o treinamento. E Oliveira fecha com a inferência, as aplicações e as limitações.
>
> ▶ O carimbo no rodapé mostra sempre quem está falando. E estas marcas na linha do tempo são os momentos em que um código roda ao vivo. Começo pelo problema que motivou tudo isso.

Referências: não se aplica.

C02 · Por que não bastavam as RNNs

C02-S01 · A leitura em série

`01:25–02:20 · 00:55`

Objetivo: mostrar que a RNN processa uma posição por vez e comprime o passado em um estado.

NA TELA

Corrente de seis células em linha, com as palavras de uma frase de seis tokens abaixo. Equação: $h_t = f(h_{t-1}, x_t)$. Um relógio de operações: uma barra por passo, que não pode ser paralelizada.

CLIQUES

1. Token 1 entra e a célula 1 acende; a seta h_1 é desenhada até a célula 2. Repete até o token 6, com ritmo constante, para o espectador sentir a espera.
2. O vetor h_t, sempre do mesmo tamanho, é mostrado sobre cada seta: a largura não cresce, embora o texto cresça.
3. As barras se empilham em série; uma nota vermelha indica "passo t depende de $t-1$".

> ### TEXTO FALADO
>
> Antes do Transformer, o padrão para texto eram as redes recorrentes, as RNNs. Uma RNN lê como nós: uma palavra depois da outra, atualizando uma memória. A equação diz isso: o estado h no instante t é uma função do estado anterior, h de t menos um, e da entrada atual, x de t.
>
> ▶ Reparem que o vetor de estado tem sempre o mesmo tamanho. A frase cresce, mas a memória não. Tudo o que foi lido precisa caber num vetor fixo.
>
> ▶ E há uma consequência de engenharia: como cada passo depende do anterior, os cálculos dentro de uma frase não podem rodar em paralelo. O passo t espera o passo t menos um. E há um problema mais sério: treinar isso por muitos passos.

Referências: Cho et al. (2014); Sutskever et al. (2014).

C02-S02 · Gradiente de longo alcance

`02:20–03:20 · 01:00`

Objetivo: explicar por que dependências longas são difíceis de aprender e o que a LSTM resolve de fato.

NA TELA

A corrente da cena anterior sobe. Equação centralizada: $\frac{\partial h_T}{\partial h_k} = \prod_{t=k+1}^{T} \frac{\partial h_t}{\partial h_{t-1}}$. Gráfico de $0{,}9^n$ e $1{,}1^n$ em escala logarítmica. Célula LSTM em esquema, com a linha do estado de célula passando quase reta.

CLIQUES

1. Uma seta de gradiente percorre a corrente da direita para a esquerda; sua espessura diminui a cada célula.
2. As duas curvas são desenhadas. Rótulos aparecem nos pontos reais: $0{,}9^{50} \approx 0{,}005$ e $1{,}1^{50} \approx 117$.
3. A célula LSTM entra e o caminho do estado de célula é destacado em azul.

> ### TEXTO FALADO
>
> Para treinar, o erro precisa voltar do fim da frase até o começo. Pela regra da cadeia, o gradiente é um produto de muitas matrizes jacobianas, uma por passo. E um produto longo tende a sumir ou a explodir.
>
> ▶ Vejam com números simples. Um fator de 0,9 repetido cinquenta vezes dá cerca de 0,005: o sinal praticamente desaparece. Um fator de 1,1 repetido cinquenta vezes dá cerca de 117: o sinal explode. Por isso dependências longas são tão difíceis de aprender.
>
> ▶ A LSTM ataca isso com portas e com um estado de célula que se atualiza de forma quase aditiva. Esse caminho, em azul, deixa o gradiente passar com bem menos perda. Mas atenção: a LSTM atenua o problema, não elimina. E continua lendo em série. O outro problema era o funil: uma frase inteira em um único vetor.

Referências: Bengio; Simard; Frasconi (1994); Hochreiter; Schmidhuber (1997); Pascanu; Mikolov; Bengio (2013).

C02-S03 · O gargalo e a atenção de Bahdanau

`03:20–04:35 · 01:15`

Objetivo: mostrar a atenção como solução para o gargalo do seq2seq e como ponte para o Transformer.

NA TELA

À esquerda, encoder com estados $h_1, \ldots, h_n$ em coluna. À direita, decoder com s_{i-1}. No centro, primeiro um único vetor c como funil; depois, linhas de cota de cada h_j até o decoder. Equações e entrando uma a uma.

CLIQUES

1. Todos os h_j convergem para um único c; o funil é desenhado em vermelho.
2. O funil é apagado; linhas finas ligam cada h_j ao decoder.
3. A espessura das linhas muda conforme α_{ij}, usando uma tradução curta como exemplo.
4. Entram as equações, com e_{ij}, α_{ij} e c_i destacados na ordem de uso.

> #### TEXTO FALADO
>
> Na tradução com redes recorrentes, o chamado seq2seq, o encoder lê a frase de origem e entrega ao decoder um único vetor de contexto, c. Esse funil é o gargalo: quanto mais longa a frase, mais informação se perde.
>
> *(Pausa de 1 s.)*
>
> ▶ Em 2014, Bahdanau, Cho e Bengio propõem outra coisa: a cada palavra gerada, o decoder calcula um contexto próprio, olhando para todos os estados do encoder.
>
> ▶ A espessura de cada linha é o peso que aquele estado recebe na palavra atual.
>
> ▶ São três equações. Primeiro, e i j é a pontuação de compatibilidade entre o estado do decoder e cada estado h j. Ela é calculada por uma pequena rede: é a atenção aditiva. Depois, alfa i j aplica a exponencial e divide pela soma, transformando pontuações em pesos. Por fim, c i é a média dos h j ponderada por esses pesos. Guardem essa receita: pontuar, normalizar, fazer a média. Ela vai voltar. Mas reparem que os h j ainda vêm de uma RNN.

Referências: Bahdanau; Cho; Bengio (2014); Sutskever et al. (2014).

C02-S04 · A pergunta-ponte

`04:35–05:00 · 00:25`

Objetivo: formular a pergunta que o Transformer responde.

NA TELA

A prancha esvazia. No centro: "E se retirarmos a recorrência?". Abaixo, a referência: Vaswani et al., 2017.

CLIQUES

1. A RNN da cena anterior é riscada com um traço vermelho e sai, ficando só as linhas de atenção.
2. A pergunta entra e permanece parada.

> #### TEXTO FALADO
>
> Se a atenção já decide o que importa em cada momento, para que manter a recorrência?
>
> ▶ *(Deixar a pergunta 3 s na tela, em silêncio.)*Em 2017, um grupo do Google responde com o artigo "Attention Is All You Need". Mas, para calcular atenção, o texto precisa virar números.

Referências: Vaswani et al. (2017).

C03 · Tokenização e embeddings

C03-S01 · Por que tokens

`05:00–05:30 · 00:30`

Objetivo: justificar a unidade intermediária entre caractere e palavra.

NA TELA

Três fileiras horizontais da mesma frase, por caracteres, por palavras, por subpalavras. À direita de cada uma, dois números: comprimento da sequência e tamanho típico do vocabulário, em ordem de grandeza.

CLIQUES

1. Cada fileira se divide em peças.
2. Os problemas de cada uma são marcados em vermelho: "sequência longa" e "palavra fora do vocabulário". A fileira de subpalavras recebe um traço azul.

> #### TEXTO FALADO
>
> Primeira decisão: qual é a unidade do texto? Por caracteres, o vocabulário é pequeno, mas as sequências ficam muito longas. Por palavras, as sequências são curtas, mas o vocabulário é enorme, e uma palavra nova fica sem representação.
>
> ▶ Subpalavras são o meio-termo. Por isso, token não é sinônimo de palavra. Como escolher essas subpalavras? Vamos executar.

Referências: Sennrich; Haddow; Birch (2016).

C03-S02 · BPE ao vivo e WordPiece

`05:30–06:55 · 01:25`

Objetivo: mostrar, com código real, como o vocabulário surge de fusões frequentes.

NA TELA

A demo em tela cheia. Corpus com segmentação atual. Pares mais frequentes. Lista de fusões e campo de palavra nova. Ao final, nota de WordPiece.

CLIQUES

A própria demo. Depois da demo, peças de "playing" entram como `play` e `##ing` para contrastar com o marcador `</w>`.

> #### DEMONSTRAÇÃO
>
> `demos/00-tokenizacao-bpe`. Roteiro de cliques:
>
> 1. Mostrar o corpus: gato 5×, gatos 3×, pato 6×, patos 2×, gatinho 2×, patinho 1×, mato 3×.
> 2. Clicar em "Próxima fusão": a+t, com 22 ocorrências.
> 3. Mais dois cliques: at+o (19) e ato+`</w>` (14).
> 4. Mais dois cliques: p+ato`</w>` (6) e ato+s (5).
> 5. No campo, deixar "ratos": a palavra nunca vista vira `r`, `atos`, `</w>`.
> 6. Contingência: `roteiro/previews/demo-00-bpe.png`.

> #### TEXTO FALADO
>
> Este é o BPE, Byte Pair Encoding, implementado aqui. Ele começa com caracteres e um marcador de fim de palavra. O corpus tem "gato" cinco vezes, "pato" seis vezes, "mato" três vezes e algumas variações.
>
> *(Clicar em "Próxima fusão".)* O algoritmo conta os pares de símbolos vizinhos, ponderados pela frequência de cada palavra, e funde o par mais frequente. Aqui, "a" com "t", com vinte e duas ocorrências.
>
> *(Clicar mais duas vezes.)* Ele repete: "at" com "o", depois "ato" com o fim de palavra.
>
> *(Clicar mais duas vezes.)* Agora "p" com "ato", e "ato" com "s".
>
> *(Escrever "ratos" no campo.)* A palavra "ratos" nunca apareceu no corpus, e mesmo assim é coberta: "r", mais "atos", mais o fim de palavra. É assim que palavras raras ganham representação. A lista de fusões, na ordem em que foram feitas, é o próprio tokenizador. Aplicada a um texto novo, ela gera os tokens.

▶ O WordPiece, usado no BERT, tem a mesma ideia de subpalavras, mas escolhe as peças por outro critério e marca a continuação com duas cerquilhas, como em "play" e "cerquilha cerquilha ing". Agora temos números inteiros. E uma rede neural precisa de vetores.

Referências: Sennrich; Haddow; Birch (2016); Schuster; Nakajima (2012); Wu et al. (2016); Devlin et al. (2018).

C03-S03 · Do token ao vetor

`06:55–08:00 · 01:05`

Objetivo: apresentar a matriz de embeddings como uma tabela aprendida.

NA TELA

Matriz $E \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$ desenhada como tabela alta, com uma linha por token. Token → id → linha → vetor. Plano com pontos de tokens próximos e distantes, rotulado como projeção 2D ilustrativa.

CLIQUES

1. O id do token aponta para a linha da matriz, e ela é copiada para a direita como vetor com células coloridas por dimensão.
2. Repetir com um segundo token, mais rápido.
3. Os pontos aparecem no plano; linhas de cota mostram a proximidade de tokens de uso parecido.

TEXTO FALADO

Cada token tem um número, o id, e esse id aponta para uma linha de uma tabela: a matriz de embeddings, com uma linha por token do vocabulário. Consultar o embedding é só selecionar essa linha.

▶ Com outro token, a mesma coisa: outra linha, outro vetor. No artigo original, cada vetor tem 512 dimensões, o chamado d model. Os valores começam aleatórios e são aprendidos durante o treino.

▶ Com o treino, tokens usados de modo parecido tendem a ficar próximos, porque isso ajuda a prever o próximo token. Este plano é uma projeção em duas dimensões, só para ilustrar. E um cuidado: uma dimensão isolada não tem significado garantido. O que importa é a geometria do conjunto, as distâncias e as direções entre os vetores. Mas essa tabela tem um limite grave.

Referências: Vaswani et al. (2017); Jurafsky; Martin (2026).

C03-S04 · O problema do banco

`08:00–08:45 · 00:45`

Objetivo: mostrar que o embedding é igual em qualquer contexto e passar a palavra.

NA TELA

Duas frases, uma sobre a outra, "o banco aprovou o crédito" e "sentei no banco da praça". As duas ocorrências de "banco" são destacadas. Dois vetores idênticos, célula por célula, com um sinal de igual em vermelho.

CLIQUES

1. As frases entram.
2. Da palavra "banco" de cada frase desce uma seta até um vetor.
3. O sinal de igual entra, seguido da frase "A tabela não vê os vizinhos".
4. O carimbo troca de nome como um carimbo real: desce, encosta, volta.

TEXTO FALADO

Leiam estas duas frases: "o banco aprovou o crédito" e "sentei no banco da praça". Na primeira, banco é uma instituição financeira. Na segunda, é um assento.

▶ Mas, pela tabela, o vetor de "banco" é o mesmo nas duas.

▶ Idêntico, célula por célula. *(Pausa de 2 s.)* O significado está nos vizinhos, e a tabela não vê os vizinhos. Precisamos de um mecanismo que recalcule o vetor de cada token em função do contexto.

▶ Davi, como fazer um token olhar para os outros?

Referências: Vaswani et al. (2017).

Davi

Mecanismo de atenção · de 08:45 a 17:30

C04 · Self-attention

C04-S01 · Reponderar pelo contexto

08:45–09:25 · 00:40

Objetivo: dar a ideia central antes da matemática e declarar os limites da metáfora.

NA TELA

Frase "o banco aprovou o crédito" em linha, como peças. Linhas de cota saem de "banco" para todas as peças, inclusive para ela mesma. A fórmula-resumo: $z_i = \sum_j \alpha_{ij}\, v_j$.

CLIQUES

1. Peças entram uma a uma.
2. Cotas finas são desenhadas de "banco" até todas.
3. As cotas engrossam até os pesos finais. Um vetor novo, em verde, surge sob "banco".
4. Texto "limites da metáfora" entra em vermelho e em corpo pequeno.

> TEXTO FALADO
>
> A ideia central é simples.
>
> ▶ O token "banco" olha para todos os tokens da frase, inclusive para ele mesmo.
>
> ▶ O vetor novo de "banco" é uma média ponderada do que os outros têm a oferecer. É a mesma receita de Bahdanau, agora dentro da própria frase. Na nossa prancha, a atenção é a cota: liga as peças, e a espessura indica o peso.
>
> ▶ Mas a metáfora tem limites: aqui o vetor muda de verdade, e um peso alto não prova que o modelo entendeu alguma coisa. Média ponderada de quê, e com quais pesos? Três matrizes respondem.

Referências: Vaswani et al. (2017); Jain; Wallace (2019).

C04-S02 · Q, K e V

09:25–10:25 · 01:00

Objetivo: definir consulta, chave e valor como projeções lineares aprendidas do mesmo X.

NA TELA

Matriz X de $n \times d_{\text{model}}$ à esquerda. Três setas para W^Q, W^K, W^V no centro. Três matrizes à direita, em azul, vermelho e ocre. Na parte de baixo, analogia em três palavras: pergunta, etiqueta, conteúdo.

CLIQUES

1. X entra com rótulo de dimensão.
2. Três setas são desenhadas, uma por vez, cada uma com a sua cor.
3. As matrizes Q, K, V surgem por varredura de linha.
4. Um token em foco: sua linha acende nas três matrizes.

> TEXTO FALADO
>
> Partimos da matriz X: uma linha por token, e cada linha com d model dimensões.
>
> ▶ Desse mesmo X saem três projeções. São só multiplicações por matrizes aprendidas: Q é X vezes W Q, K é X vezes W K, e V é X vezes W V.
>
> ▶ Pensem numa consulta a um arquivo. A consulta, Q, diz o que eu procuro. A chave, K, diz o que cada ficha oferece. E o valor, V, é o conteúdo que vai ser usado.

▶ Cada token tem a sua linha nas três matrizes. A diferença crucial: num arquivo comum, você pega uma ficha só. Aqui, você toma um pouco de todas, na proporção da compatibilidade. E, como as três vêm da mesma sequência, isso se chama self-attention. Como medir o quanto uma consulta combina com uma chave?

Referências: Vaswani et al. (2017); Jurafsky; Martin (2026).

C04-S03 · Produto escalar como compatibilidade

`10:25–11:05 · 00:40`

Objetivo: justificar o produto escalar como pontuação e sua forma matricial.

NA TELA

Plano com um vetor q azul e duas chaves vermelhas, uma alinhada e outra ortogonal, com o ângulo marcado. Cálculo por extenso de $q_{\text{gato}} \cdot k_{\text{dorme}}$ e, abaixo, a matriz $QK^\top$.

CLIQUES

1. Os vetores são desenhados; o arco do ângulo aparece.
2. Os pares de componentes se multiplicam e somam: $0 \cdot 0 + 2 \cdot 2 + 1 \cdot 1 + 0 \cdot 0 = 5$.
3. A linha de Q cruza a coluna de $K^\top$, e o 5 pousa em uma célula de $QK^\top$.

TEXTO FALADO

Com o produto escalar. Ele soma os produtos componente a componente, e é igual ao produto dos comprimentos vezes o cosseno do ângulo. É grande quando os vetores apontam para o mesmo lado, e zero quando são perpendiculares.

▶ No exemplo: zero vezes zero, mais dois vezes dois, mais um vezes um, mais zero vezes zero. Dá cinco.

▶ Na forma de matriz, Q vezes K transposta calcula todos os pares de uma vez. Isso substitui a pequena rede de Bahdanau e roda em código de matrizes muito otimizado. Mas há uma armadilha quando os vetores são grandes.

Referências: Vaswani et al. (2017).

C04-S04 · Por que dividir por raiz de d_k

`11:05–12:10 · 01:05`

Objetivo: explicar com dados que a escala controla a variância das pontuações e evita a saturação do softmax.

NA TELA

Demo aberta na aba 3. Curvas de maior peso em função de d_k. Barras de um sorteio. A equação em destaque: $\mathrm{Var}(q \cdot k) = d_k$.

CLIQUES

Antes de abrir a demo, a dedução em três linhas:

1. Componentes independentes, média 0, variância 1;
2. $q \cdot k$ tem média 0 e variância d_k;
3. Dividir por $\sqrt{d_k}$ devolve variância 1.

DEMONSTRAÇÃO

`demos/01-self-attention/?aba=escala&cena=C04-S04`. Mover o controle de d_k de 4 até 1024 e parar em 64. Contingência: `roteiro/previews/demo-01-escala.png`.

TEXTO FALADO

O artigo supõe componentes independentes, com média zero e variância um.

▶ Então o produto escalar de q e k tem média zero e variância d k. Quanto maior a dimensão, mais espalhadas ficam as pontuações.

▶ Dividir por raiz de d k devolve a variância para um.

▶ Na demo, eu vou de d k igual a 4 até 1024 e paro em 64. *(Mover o controle e parar em 64. Pausa de 2 s.)* Sem a escala, o maior peso médio sobe em direção a 1: o softmax satura, e quase todo o peso vai para um token só. Com a escala, fica estável. Por que isso importa? A derivada de um peso em relação à sua pontuação é p vezes um menos p. Se p é quase 1, o gradiente quase some, e o modelo para de aprender. Um cuidado: dividir por raiz de d k não faz os pesos somarem 1. Quem faz isso é o softmax. Por falar nele.

Referências: Vaswani et al. (2017), seção 3.2.1 e nota 4.

C04-S05 · Softmax e média ponderada

`12:10–12:40 · 00:30`

Objetivo: fechar a fórmula completa do artigo.

NA TELA

Equação central, $\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$. Abaixo de cada termo, um rótulo: compatibilidade, escala, pesos por linha, média dos valores.

CLIQUES

1. A equação entra termo a termo, com os demais a 40%.
2. Rótulos ligados por cotas aos termos.

TEXTO FALADO

O softmax aplica a exponencial a cada pontuação e divide pela soma da linha. Cada linha vira um conjunto de pesos positivos que somam 1. Multiplicar por V faz a média ponderada dos valores.

▶ A fórmula inteira diz: compare consultas e chaves, divida pela raiz de d k, transforme em pesos e faça a média dos valores. Vamos fazer essa conta à mão.

Referências: Vaswani et al. (2017), equação 1.

C04-S06 · A conta à mão, passo a passo

`12:40–14:20 · 01:40`

Objetivo: executar um exemplo pequeno que o espectador pode conferir com calculadora.

NA TELA

Entradas: tokens "o", "gato", "dorme", e as matrizes abaixo. Etapas em quatro colunas, como em `roteiro/previews/demo-01-mao.png`.

$$Q = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 2 & 1 & 0 \\ 2 & 1 & 0 & 0 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 2 & 0 & 1 & 1 \\ 0 & 2 & 1 & 0 \end{bmatrix}, \quad V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \quad d_k = 4$$

CLIQUES

Um passo por clique. Os valores são os mesmos conferidos em `demos/01-self-attention/teste_referencia.py`.

1. Entram Q, K, V, mostrando as dimensões.
2. Produto: a linha de "gato" cruza as colunas, resultando em $QK^\top = \begin{bmatrix} 1 & 3 & 0 \\ 0 & 1 & 5 \\ 2 & 4 & 2 \end{bmatrix}$.
3. Escala: cada número é dividido por $\sqrt{4} = 2$, e os valores se transformam no lugar: $\begin{bmatrix} 0{,}50 & 1{,}50 & 0{,}00 \\ 0{,}00 & 0{,}50 & 2{,}50 \\ 1{,}00 & 2{,}00 & 1{,}00 \end{bmatrix}$.
4. Softmax linha a linha, com a soma "= 1" aparecendo à direita de cada linha: $A = \begin{bmatrix} 0{,}231 & 0{,}629 & 0{,}140 \\ 0{,}067 & 0{,}111 & 0{,}821 \\ 0{,}212 & 0{,}576 & 0{,}212 \end{bmatrix}$.

5. Média ponderada, com a linha de "gato" em foco: $z_{\text{gato}} = 0{,}067\,(1,0) + 0{,}111\,(0,1) + 0{,}821\,(1,1) = (0{,}889;\ 0{,}933)$. Resultado em verde: $Z = AV = \begin{bmatrix} 0{,}371 & 0{,}769 \\ 0{,}889 & 0{,}933 \\ 0{,}424 & 0{,}788 \end{bmatrix}$.
6. Sobre a frase, as linhas de cota aparecem com as espessuras de A.

> **DEMONSTRAÇÃO**
> A aba 1 de **`demos/01-self-attention`** tem a mesma conta. Pode substituir a animação em caso de falha: **`?aba=mao`**.

> **TEXTO FALADO**
> Três tokens: "o", "gato" e "dorme". Consultas e chaves com quatro dimensões, valores com duas. Os números foram escolhidos para a conta caber na tela. Não são pesos aprendidos.
>
> ▶ Primeira etapa: o produto Q vezes K transposta. Acompanhem a linha de "gato". Com a chave de "o", dá zero. Com a de "gato", um. Com a de "dorme", dois vezes dois mais um vezes um: cinco.
>
> ▶ Segunda etapa: dividir tudo pela raiz de 4, que é 2. A linha de "gato" fica 0; 0,5; e 2,5.
>
> ▶ Terceira etapa: softmax, linha por linha. Na linha de "gato", as exponenciais de 0, 0,5 e 2,5 valem cerca de 1; 1,65; e 12,18. A soma é cerca de 14,83. Dividindo cada uma pela soma: 0,067; 0,111; e 0,821. E cada linha soma 1.
>
> ▶ Quarta etapa: a média dos valores com esses pesos. Para "gato", o resultado é 0,889 e 0,933.
>
> ▶ Sobre a frase, as cotas mostram os mesmos pesos. *(Pausa de 2 s.)*A saída de cada token agora mistura informação dos outros, e cada linha de pesos soma 1. Agora, com o código rodando.

Referências: Vaswani et al. (2017), equação 1.

C04-S07 · Mapa de calor ao vivo

`14:20–15:45 · 01:25`

Objetivo: mostrar o código funcionando e resolver o problema do banco deixado por Palos.

NA TELA

Demo na aba 2. Linhas de cota e mapa de calor. Tabela da consulta em foco, plano finanças por lugar e nota vermelha de que os vetores foram escritos à mão.

CLIQUES

A própria demo. Antes dela, o código de **`atencao.js`** sobre papel, destacando a linha do softmax; o passo seguinte abre a demo.

> **DEMONSTRAÇÃO**
> **`demos/01-self-attention/?aba=frase`**.
>
> 1. Passar o mouse por "banco" na frase "o banco aprovou o crédito": maior peso em "crédito", 0,421; saída $Z = (0{,}651;\ 0{,}093)$, deslocada para finanças.
> 2. Passar por outros tokens para mostrar que cada linha é uma distribuição.
> 3. Clicar em "Trocar frase": em "sentei no banco da praça", a mesma consulta dá 0,412 para "praça", e $Z = (0{,}091;\ 0{,}647)$, deslocada para lugar.
> 4. Alternar "Dividir por √d_k" para ver as linhas ficarem mais concentradas.
> 5. Contingência: **`roteiro/previews/demo-01-frase.png`** e **`demo-01-frase-praca.png`**.

> **TEXTO FALADO**
> Este é o código da atenção, em JavaScript puro. São poucas linhas, e um teste compara o resultado com o NumPy e com a conta que acabamos de fazer à mão. A linha destacada é o softmax.
>
> ▶ *(Passar o mouse sobre "banco".)*Na frase "o banco aprovou o crédito", quando "banco" é a consulta, o maior peso vai para "crédito": 0,421. E a saída se desloca para o lado de finanças.

> *(Passar o mouse por outros tokens.)* Cada linha é uma distribuição de pesos que soma 1.
>
> *(Clicar em "Trocar frase".)* Agora, "sentei no banco da praça". Os vetores de "banco" são exatamente os mesmos do dicionário. Só o contexto mudou. A mesma consulta dá 0,412 para "praça", e a saída vai para o lado de lugar. Essa é a resposta ao problema do banco que o Palos deixou.
>
> *(Alternar "Dividir por √d_k".)* Sem a divisão, as linhas ficam mais concentradas. Uma ressalva: estes vetores foram escritos à mão. Num modelo real, W Q, W K e W V são aprendidas. E um conjunto de pesos só permite um padrão de atenção por vez. E se quisermos vários padrões ao mesmo tempo?

Referências: Vaswani et al. (2017).

C05 · Multi-head attention

C05-S01 · Várias projeções em paralelo

`15:45–16:40 · 00:55`

Objetivo: explicar a fórmula de multi-head e por que o custo não multiplica.

NA TELA

Uma barra de 512 dimensões depois dividida em 8 segmentos de 64. Oito pequenos mapas de calor em fila. Concatenação e W^O. Equações:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O$$

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

CLIQUES

1. A barra é cortada por 7 traços verticais.
2. Cada segmento desce e vira um mapa de calor, todos ao mesmo tempo.
3. As saídas se juntam em uma barra e passam por W^O.

> ### TEXTO FALADO
>
> Uma única média ponderada tende a borrar relações diferentes. Em vez disso, o Transformer faz várias atenções em paralelo. No artigo, d model é 512 e há oito cabeças.
>
> ▶ Cada cabeça trabalha com 64 dimensões: d k e d v são 512 dividido por 8. Cada uma tem as suas próprias matrizes W Q, W K e W V e calcula o seu próprio mapa de atenção. Estes mapas usam valores de ilustração. Como cada cabeça tem dimensão reduzida, o custo total fica parecido com o de uma única cabeça de dimensão completa.
>
> ▶ As saídas das cabeças são concatenadas e passam por W O, que devolve o tamanho original e mistura as cabeças. O que cada cabeça aprende? Essa pergunta exige cuidado.

Referências: Vaswani et al. (2017), seção 3.2.2.

C05-S02 · O que cada cabeça aprende

`16:40–17:10 · 00:30`

Objetivo: dar a intuição sem afirmar que as cabeças têm funções definidas de antemão.

NA TELA

A mesma frase repetida em três vistas, como vistas ortogonais de uma peça: frontal, lateral, superior. Cada uma com linhas de cota diferentes. Nota vermelha: "padrões observados, não programados".

CLIQUES

1. As três vistas se desenham em sequência.
2. A nota entra por último.

> ### TEXTO FALADO
>
> Pensem nas vistas de uma peça: frontal, lateral e superior. Cada projeção revela relações diferentes. Em modelos treinados, pesquisadores observam cabeças que tendem a seguir certos padrões, por exemplo

sintáticos.

▶ Mas ninguém programa isso, e o padrão não é garantido. Peso de atenção não é explicação do modelo. Temos a peça central. Falta um detalhe que ela perdeu.

Referências: Vaswani et al. (2017); Clark et al. (2019); Jain; Wallace (2019).

C05-S03 · A peça está pronta, falta a ordem

`17:10–17:30 · 00:20`

Objetivo: sintetizar o bloco de atenção e passar a palavra.

NA TELA

Caixa desenhada "Multi-head attention" no centro, com entradas Q, K, V e saída. Do lado esquerdo, uma mini-planta do modelo com essa peça preenchida e as demais tracejadas.

CLIQUES

1. A caixa se fecha em torno dos mapas de calor da cena anterior.
2. A mini-planta entra com a peça preenchida em azul e as demais tracejadas.
3. Carimbo troca de nome.

TEXTO FALADO

Recapitulando: pontuar, escalar, normalizar e fazer a média, em várias cabeças.

▶ Agora reparem: em nenhuma dessas contas aparece a ordem das palavras.

▶ Mendes, como esse modelo sabe quem veio antes de quem?

Referências: Vaswani et al. (2017).

Mendes

Arquitetura e treinamento · de 17:30 a 26:20

C06 · Codificação posicional

C06-S01 · Atenção ignora a ordem

17:30–18:05 · 00:35

Objetivo: mostrar que a self-attention, sozinha, é equivariante à permutação.

NA TELA

"o gato dorme" com sua matriz A. "dorme gato o" com sua matriz. A afirmação $f(PX) = Pf(X)$, onde P é uma matriz de permutação.

CLIQUES

1. A matriz A do exemplo à mão entra à esquerda.
2. Os tokens trocam de lugar por um arco; linhas e colunas se reordenam junto.
3. Linhas de ligação mostram que cada valor é o mesmo, só em outra posição.

> TEXTO FALADO
>
> Esta é a matriz de atenção de "o gato dorme", a mesma da conta à mão.
>
> ▶ Agora eu embaralho os tokens: "dorme gato o". As linhas e as colunas trocam de lugar junto.
>
> ▶ E cada valor é o mesmo, só em outra posição. Nada na fórmula sabe o que é "antes". Para o modelo, "o cachorro mordeu o homem" e "o homem mordeu o cachorro" teriam os mesmos ingredientes. A posição precisa entrar pelos dados. A solução do artigo original é somar uma régua de frequências.

Referências: Vaswani et al. (2017), seção 3.5.

C06-S02 · Curvas senoidais ao vivo

18:05–19:25 · 01:20

Objetivo: visualizar a fórmula e entender por que frequências diferentes dão uma assinatura para cada posição.

NA TELA

A demo em tela cheia. Quatro curvas, com dimensão e comprimento de onda à esquerda. Equações, matriz PE e similaridade entre posições.

CLIQUES

Antes da demo, uma linha: embedding + PE = entrada, com as duas barras se somando célula a célula.

> DEMONSTRAÇÃO
>
> `demos/02-codificacao-posicional`.
>
> 1. Curvas com $d_{\text{model}} = 128$: as dimensões 0, 16, 32 e 48 têm λ de cerca de 6,3; 19,9; 62,8 e 198,7 posições.
> 2. Mover o mouse na matriz: a linha de posição percorre o mapa, e os pontos andam nas curvas.
> 3. Mostrar a similaridade: posições vizinhas têm vetores mais parecidos.
> 4. Mudar d_{model} para mostrar o efeito na matriz.
> 5. Contingência: `roteiro/previews/demo-02-posicional.png`.

> TEXTO FALADO
>
> Cada token recebe um vetor de posição, que é somado ao embedding, não concatenado. Por isso ele tem a mesma dimensão, d model.
>
> ▶ As fórmulas: nas dimensões pares, o seno da posição dividida por dez mil elevado a dois i sobre d model. Nas ímpares, o cosseno da mesma expressão. Na demo, cada curva é uma dimensão, e cada uma oscila numa frequência. A dimensão zero se repete a cada 6,3 posições; a dimensão 48, só a cada 199. Os comprimentos

de onda vão de dois pi a dez mil vezes dois pi, em progressão geométrica. É como os ponteiros de um relógio, ou os bits de um contador, só que contínuos.

*(Mover o mouse sobre a matriz.)*Cada posição ganha uma assinatura própria. E posições vizinhas têm vetores parecidos.

*(Mudar o d model.)*O artigo cita uma vantagem: para um deslocamento fixo k, a codificação da posição mais k é uma função linear da codificação da posição. Isso pode facilitar aprender posições relativas, e o teste da demo confere essa identidade. E aqui não há nenhum parâmetro para treinar. Mas essa não é a única solução.

Referências: Vaswani et al. (2017), seção 3.5.

C06-S03 · Posições aprendidas e RoPE

19:25–19:55 · 00:30

Objetivo: mostrar as alternativas à versão senoidal, em nível de visão geral.

NA TELA

Três colunas iguais, senoidal (fixa, somada à entrada), aprendida (tabela de posições, somada à entrada), RoPE (rotação de Q e K na atenção). Cada coluna com um desenho pequeno e duas linhas de texto.

CLIQUES

1. As colunas entram da esquerda para a direita.
2. Na coluna RoPE, dois vetores são redesenhados com o mesmo giro somado aos dois, e o ângulo entre eles permanece igual.

TEXTO FALADO

Há duas alternativas comuns. Posições aprendidas, usadas no BERT e no GPT: uma tabela como a de embeddings, limitada ao tamanho máximo definido no treino.

▶ E o RoPE, que gira pares de dimensões de Q e K conforme a posição. Assim, o produto escalar passa a depender da distância entre os tokens, e não da posição absoluta. Vários modelos recentes usam essa ideia. Com a posição resolvida, podemos montar o bloco.

Referências: Gehring et al. (2017); Devlin et al. (2018); Radford et al. (2018); Su et al. (2024).

C07 · O bloco completo

C07-S01 · Residuais e LayerNorm

19:55–20:40 · 00:45

Objetivo: explicar as duas peças que tornam possível treinar pilhas profundas.

NA TELA

Caixa de atenção no centro. A linha residual contorna a caixa por fora, como cota de desvio. Bloco "Add & Norm" depois da soma. $\mathrm{LayerNorm}(x + \mathrm{Sublayer}(x))$. Uma barra de um vetor, antes e depois da normalização.

CLIQUES

1. O vetor atravessa a caixa.
2. A linha residual é desenhada por fora e se soma à saída.
3. As células do vetor se reajustam para média 0 e variância 1 na normalização.
4. Entra a forma pre-norm, $x + \mathrm{Sublayer}(\mathrm{LayerNorm}(x))$, abaixo da forma original.

TEXTO FALADO

O vetor atravessa a subcamada de atenção.

▶ A conexão residual soma a entrada à saída. É um caminho direto, que ajuda o gradiente a chegar às camadas iniciais.

▶ Depois vem a LayerNorm, que normaliza cada posição ao longo das suas features, para média zero e variância um, sem depender do lote.

▶ Um detalhe que costuma ser confundido: o artigo aplica a normalização depois da soma, o post-norm. O GPT-2 e muitos modelos posteriores a colocam antes, o pre-norm. O nosso mini-GPT usa pre-norm. A atenção só faz média. Onde está a não linearidade forte?

Referências: He et al. (2016); Ba; Kiros; Hinton (2016); Vaswani et al. (2017); Radford et al. (2019).

C07-S02 · FFN por posição

20:40–21:10 · 00:30

Objetivo: mostrar a rede aplicada igualmente a cada posição.

NA TELA

Coluna de vetores à esquerda. No centro, uma única FFN desenhada, ligada a todas as posições. À direita, a saída. Equação: $\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$. Rótulo: $512 \to 2048 \to 512$.

CLIQUES

1. A FFN recebe a primeira posição; o vetor se expande e contrai.
2. A mesma FFN processa as demais, com um mesmo traço ligando todas.

TEXTO FALADO

Na rede feed-forward, aplicada a cada posição. São duas camadas lineares com uma ReLU no meio. No artigo, o vetor vai de 512 para 2048 dimensões e volta para 512.

▶ A mesma rede processa todas as posições: os pesos são iguais dentro de uma camada e mudam de uma camada para outra. Empilhando tudo, temos a arquitetura completa.

Referências: Vaswani et al. (2017), seção 3.3.

C07-S03 · Encoder e decoder

21:10–21:50 · 00:40

Objetivo: apresentar o diagrama completo e distinguir encoder, decoder, encoder-only e decoder-only.

NA TELA

Planta do modelo, desenhada como esquema: pilha de encoder à esquerda ($N = 6$) e de decoder à direita. Tabela pequena com a arquitetura usada por família: Transformer original: encoder e decoder; BERT: só encoder; GPT: só decoder, sem cross-attention.

CLIQUES

1. As caixas já vistas entram na planta, uma por uma. A única nova é a cross-attention, ainda tracejada.
2. O multiplicador $\times N$ é cotado ao lado de cada pilha.
3. As linhas da tabela entram, com cada uma destacando a parte da planta que usa.

TEXTO FALADO

Esta é a planta do Transformer original. O encoder tem self-attention e feed-forward. O decoder tem três subcamadas: atenção mascarada, cross-attention e feed-forward.

▶ No artigo, cada pilha repete o bloco seis vezes. A tarefa original era tradução: o encoder lê a frase de origem, e o decoder gera a de destino.

▶ As famílias usam partes dessa planta. O BERT usa só o encoder. O GPT é chamado de decoder-only porque tem a máscara causal, mas não tem a cross-attention. Por que o decoder precisa de máscara? Vamos ver.

Referências: Vaswani et al. (2017), seção 3.1; Devlin et al. (2018); Radford et al. (2018).

C07-S04 · Máscara causal ao vivo

21:50–23:05 · 01:15

Objetivo: provar numericamente que a máscara impede o vazamento do futuro.

NA TELA

A demo em tela cheia. Duas colunas: bidirecional, e causal. Cada uma com o mapa e, abaixo, as barras de quanto cada saída mudou.

CLIQUES

A própria demo; ao abrir, a hachura vermelha é desenhada célula a célula sobre o mapa da direita.

> **DEMONSTRAÇÃO**
>
> `demos/03-mascara-causal`.
>
> 1. Mostrar os dois mapas: à direita, nada acima da diagonal.
> 2. Passar o mouse em "subiu": só vê "o", "gato" e ela mesma.
> 3. Clicar em "Trocar o token futuro" ("muro" → "telhado", posição 4).
> 4. Ler as barras. Sem máscara, todas as saídas mudam: 0,187; 1,385; 0,925; 0,065; 2,090; 2,400. Com máscara: 0; 0; 0; 0; 2,367; 2,400.
> 5. Vetores pseudoaleatórios fixos: o padrão dos pesos não tem significado linguístico. O que importa é onde são nulos.
> 6. Contingência: `roteiro/previews/demo-03-mascara.png`.

> **TEXTO FALADO**
>
> Quando o modelo treina para prever o próximo token, a resposta está na própria sequência. Sem cuidado, ele simplesmente copia do futuro. A solução: antes do softmax, somar menos infinito nas células em que j é maior que i. O softmax transforma essas células em zero exato e redistribui o peso entre as anteriores. À direita, nada acima da diagonal.
>
> *(Passar o mouse sobre "subiu".)* A palavra "subiu" só vê "o", "gato" e ela mesma.
>
> *(Clicar em "Trocar o token futuro".)* Troco "muro" por "telhado", na posição 4. Olhem as barras da direita: zero, zero, zero, zero. *(Pausa de 2 s.)* Nenhuma saída anterior mudou. Só mudam a própria posição e a seguinte. À esquerda, sem máscara, a mesma troca altera todas as posições. Esse é o comportamento desejado no encoder, que lê a frase inteira. E essa é a diferença entre a atenção bidirecional do BERT e a causal do GPT. Os vetores aqui são pseudoaleatórios: o que importa é onde os pesos são zero. No decoder original, há ainda uma terceira forma de atenção.

Referências: Vaswani et al. (2017), seção 3.2.3; Devlin et al. (2018).

C07-S05 · Cross-attention

`23:05–23:45 · 00:40`

Objetivo: mostrar de onde vêm Q, K e V na cross-attention e fechar o arco com Bahdanau.

NA TELA

Frase de origem no topo; frase de destino embaixo. Entre elas–D, linhas de cota. À direita, mapa retangular de $m \times n$, e o rótulo: Q do decoder, K e V do encoder.

CLIQUES

1. K e V descem da frase de origem, em vermelho e ocre.
2. Q sobe da de destino, em azul.
3. As cotas se formam; o mapa retangular é preenchido.
4. A caixa vazia da planta é preenchida.

> **TEXTO FALADO**
>
> Na cross-attention, a fórmula é a mesma; o que muda é a origem. As chaves e os valores descem da frase de origem, vindos do encoder.
>
> ▶ As consultas sobem da frase de destino, vindas do decoder.
>
> ▶ Por isso a matriz de pesos é retangular: m tokens de destino por n tokens de origem.

▶ Lembra alguma coisa? É a atenção de Bahdanau, agora com produto escalar e sem RNN. O arco se fecha. A máquina está montada. Como ajustar milhões de parâmetros?

Referências: Vaswani et al. (2017), seção 3.2.3; Bahdanau; Cho; Bengio (2014).

C08 · Treinamento

C08-S01 · Próximo token e masked LM

`23:45–24:35 · 00:50`

Objetivo: contrastar os dois objetivos de treino e ligar cada um à máscara correspondente.

NA TELA

Duas metades. GPT, com a sequência deslocada em uma posição. BERT, com tokens substituídos por `[MASK]`. Equação: $\mathcal{L} = -\frac{1}{|T|}\sum_{t \in T} \log p_\theta(y_t \mid \text{entrada})$, com T sendo as posições avaliadas.

CLIQUES

1. À esquerda, a sequência é copiada e deslocada; setas ligam cada entrada ao alvo seguinte.
2. À direita, alguns tokens são cobertos; setas partem dos vizinhos dos dois lados até o buraco.
3. Valores do BERT entram: 15% mascarados; desses, 80% viram `[MASK]`, 10% um token aleatório e 10% ficam iguais.

TEXTO FALADO

Há dois objetivos de treino principais. No estilo GPT, a sequência é deslocada em uma posição: cada token tem como alvo o seguinte. Por isso ele precisa da máscara causal.

▶ No estilo BERT, o masked LM: alguns tokens são escondidos, e o modelo prevê os escondidos. Como eles não estão na entrada, o modelo pode olhar para os dois lados.

▶ No BERT, 15% dos tokens são escolhidos. Desses, 80% viram MASK, 10% viram um token aleatório e 10% ficam iguais. Nos dois casos, a perda é a entropia cruzada: o modelo é penalizado quando dá pouca probabilidade ao token certo. No caso do próximo token, o que entra como contexto?

Referências: Radford et al. (2018); Devlin et al. (2018), seção 3.3.1.

C08-S02 · Teacher forcing

`24:35–25:25 · 00:50`

Objetivo: explicar por que o treino é paralelo, embora a geração seja sequencial.

NA TELA

"com teacher forcing", uma única passada com entrada verdadeira e todos os alvos. "sem ele", uma cadeia em que cada saída vira a próxima entrada e um erro se propaga. Taxa de aprendizado do artigo.

CLIQUES

1. Na linha de cima, todas as posições calculam ao mesmo tempo.
2. Na de baixo, uma por vez; um erro em vermelho contamina as seguintes.
3. Entra a taxa: $lrate = d_{\text{model}}^{-0,5} \cdot \min(step^{-0,5},\ step \cdot warmup^{-1,5})$, com $warmup = 4000$.

TEXTO FALADO

No treino, a entrada de cada posição é o token verdadeiro anterior, não o que o modelo previu. Isso se chama teacher forcing. Com a máscara causal, todas as posições são calculadas numa única passada, em paralelo.

▶ Sem isso, seria uma posição por vez, e um erro contaminaria as seguintes. Essa é a grande vantagem sobre a RNN no treino.

▶ Do artigo: otimizador Adam, taxa de aprendizado que sobe durante 4000 passos de aquecimento e depois cai, label smoothing de 0,1 e dropout de 0,1. Parece abstrato. Então treinamos um de verdade.

Referências: Williams; Zipser (1989); Vaswani et al. (2017), seção 5.

C08-S03 · Um treino real, em NumPy

25:25–26:20 · 00:55

Objetivo: mostrar que tudo o que foi descrito foi implementado e treinado, e quais números saíram.

NA TELA

Trecho do backprop de atenção em `modelo_numpy.py`. Curva real de perda de validação. Ficha técnica. Linha tracejada vermelha na perda dos bigramas.

CLIQUES

1. O código entra, com as linhas de `sbar`, `qbar` e `kbar` destacadas, uma a uma.
2. A curva é desenhada na ordem real dos passos.
3. Valores são escritos nas posições corretas: início em 3,90, bigramas em 2,36, final em 1,49.
4. O carimbo troca de nome.

DEMONSTRAÇÃO

Curva e ficha vistas em `demos/04-mini-gpt/?cena=C08-S03&apresentador=a3`, que mantém o carimbo correto; os dados vêm de `dados/treino.log`. O treino não é executado ao vivo, porque levou 24 minutos.

TEXTO FALADO

Implementamos um mini-GPT em NumPy puro, com o gradiente escrito à mão. Estas são as linhas do backprop da atenção. Conferimos tudo por diferenças finitas, e o pior erro relativo foi de cerca de três milionésimos.

▶ A ficha: 158.656 parâmetros, três blocos, quatro cabeças, dimensão 64 e contexto de 64 caracteres. O corpus tem 812 mil caracteres de Machado de Assis, em domínio público.

▶ Foram 6000 passos, cerca de 24 minutos de CPU. A perda de validação começa em 3,90 e termina em 1,49. Um modelo de bigramas fica em 2,36. O ganho vem do contexto.

▶ Esta é a curva real, lida do registro do treino.

▶ Oliveira, esse modelo está treinado. Faça-o escrever.

Referências: Vaswani et al. (2017); Radford et al. (2018); Rush (2018).

Oliveira

Inferência e aplicações · de 26:20 a 35:10

C09 · Inferência

C09-S01 · O laço autorregressivo 26:20–26:55 · 00:35

Objetivo: mostrar que gerar texto é um laço de previsão e amostragem.

NA TELA

Diagrama em ciclo: contexto → modelo → distribuição → amostra → de volta ao contexto. Código de 5 linhas ou abaixo do ciclo.

CLIQUES

1. O ciclo se desenha uma vez.
2. Três voltas rápidas, com o contexto ganhando um token a cada volta.

> TEXTO FALADO
>
> No treino, tínhamos a resposta certa. Agora não: cada novo token depende do anterior. A geração é sequencial, mesmo que o treino seja paralelo. O laço é este: calcular os logits, transformar em probabilidades, escolher um token, anexar ao contexto e repetir.
>
> ▶ A cada volta, o contexto ganha um token, até um critério de parada. A saída do modelo é uma distribuição, não uma resposta pronta. Vamos executar o modelo que acabamos de treinar.

Referências: Radford et al. (2019).

C09-S02 · Mini-GPT ao vivo e temperatura 26:55–28:05 · 01:10

Objetivo: ver a geração caractere a caractere e o efeito da temperatura.

NA TELA

A demo em tela cheia. Início do texto, saída e controle de temperatura. Barras dos 10 caracteres mais prováveis, curva de treino e ficha. Equação: $p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$.

CLIQUES

A própria demo.

> DEMONSTRAÇÃO
>
> `demos/04-mini-gpt` (funciona offline, inferência em JS puro).
>
> 1. Com "o homem que ", $T = 0{,}8$, clicar em "Gerar" e deixar correr cerca de 15 s; as barras mudam a cada caractere.
> 2. Parar, baixar para $T = 0{,}2$, gerar: repetição de trechos frequentes, como "a casa de".
> 3. Parar, subir para $T = 1{,}5$ ou $2{,}0$: caracteres improváveis e palavras inventadas.
> 4. Contingência: `?pregravado` carrega amostras gravadas no treino, para seis temperaturas; `demos/04-mini-gpt/dados/saida_pregravada.json`.

> TEXTO FALADO
>
> Tudo roda aqui no navegador, sem rede, com o modelo de 159 mil parâmetros.
>
> *(Com "o homem que " e T = 0,8, clicar em "Gerar".)* Ele gera um caractere por vez, e as barras mostram os mais prováveis a cada passo. O resultado parece português antigo, porque o corpus é Machado com a ortografia da época. Ele forma palavras e pontuação sem nenhuma regra escrita sobre isso.

> *(Parar, baixar para T = 0,2 e gerar.)* A temperatura divide os logits antes do softmax. Baixa, ela concentra a distribuição, e o texto repete trechos frequentes.
>
> *(Parar, subir para T = 1,5 e gerar.)* Alta, ela achata a distribuição, e aparecem palavras inventadas. No limite de temperatura zero, sai sempre o mais provável. Um cuidado: o modelo não sabe o que está escrevendo. Ele reproduz regularidades estatísticas, e nesta escala elas nem chegam ao sentido. Temperatura é só um dos botões.

Referências: Radford et al. (2019); Holtzman et al. (2020).

C09-S03 · Top-k e top-p

`28:05–28:50 · 00:45`

Objetivo: diferenciar corte por quantidade e corte por massa de probabilidade.

NA TELA

A demo, barras dos 16 caracteres mais prováveis, com contorno antes do corte e preenchimento depois de renormalizar. Os cortados ganham hachura vermelha.

CLIQUES

A própria demo.

> #### DEMONSTRAÇÃO
>
> `demos/05-amostragem-kv-cache`.
>
> 1. Mostrar a distribuição sem cortes.
> 2. Top-k = 5: só cinco barras seguem elegíveis; as outras recebem a hachura.
> 3. Voltar top-k para desligado e pôr top-p = 0,50: o número de elegíveis varia conforme o contexto. Mudar o texto de início para mostrar.
> 4. Contingência: `roteiro/previews/demo-05-amostragem.png`.

> #### TEXTO FALADO
>
> Esta é a distribuição do próximo caractere, sem cortes.
>
> *(Pôr top-k = 5.)* O top-k mantém os k mais prováveis, com k fixo. Aqui, só cinco seguem elegíveis; os outros ficam hachurados.
>
> *(Desligar o top-k e pôr top-p = 0,50.)* O top-p, ou amostragem de núcleo, mantém o menor conjunto cuja probabilidade somada atinge p.
>
> *(Mudar o texto de início.)* Reparem que o número de elegíveis muda com o contexto: ele acompanha a incerteza do modelo. Depois do corte, as probabilidades são renormalizadas e o token é sorteado. Nesta demo, a ordem é temperatura, top-k e top-p. Rodar esse laço tem um custo. Vejam o que estamos desperdiçando.

Referências: Fan; Lewis; Dauphin (2018); Holtzman et al. (2020).

C09-S04 · KV cache

`28:50–29:35 · 00:45`

Objetivo: mostrar que guardar K e V evita recálculo sem mudar o resultado.

NA TELA

A demo, esquema de passos, botão de comparação, dois quadros com multiplicações e tempo, e a linha de diferença entre logits.

CLIQUES

A própria demo.

> #### DEMONSTRAÇÃO
>
> `demos/05-amostragem-kv-cache/?cena=C09-S04`. Clicar em "Gerar 64 tokens das duas formas". Resultado medido: 332,8 milhões de multiplicações sem cache e 10,4 milhões com cache, 32 vezes menos. Os tempos

variam por máquina. Contingência: `roteiro/previews/demo-05-amostragem.png`.

TEXTO FALADO

Por causa da máscara causal, os tokens antigos nunca mudam quando chega um novo. As suas chaves e valores continuam valendo. Então guardamos essas matrizes e, a cada passo, calculamos só o token novo. Isso é o KV cache.

*(Clicar em "Gerar 64 tokens das duas formas". Esperar os números.)*Sem cache, 332,8 milhões de multiplicações. Com cache, 10,4 milhões: trinta e duas vezes menos. E as duas rotas dão o mesmo logit, o que o teste confere. Mas atenção: a consulta nova ainda é comparada com todas as chaves. Cada passo continua proporcional ao tamanho do contexto, e o cache ocupa memória. O que nos leva ao custo que define esta arquitetura.

Referências: Pope et al. (2022).

C10 · Custo e variantes eficientes

C10-S01 · O custo quadrático

29:35–30:05 · 00:30

Objetivo: explicar de onde vem $O(n^2)$ e quando ele pesa.

NA TELA

Três quadrados de mapa de atenção, com área proporcional, $n = 512$, $n = 1024$, $n = 2048$. Tabela de pares: 262.144; 1.048.576; 4.194.304.

CLIQUES

1. O menor quadrado é preenchido.
2. Dobrando n, a área quadruplica, e as cotas mostram isso.

TEXTO FALADO

Cada um dos n tokens é comparado com n chaves. A matriz de pontuações tem n ao quadrado células, por cabeça e por camada. Com 512 tokens, são cerca de 262 mil pares.

▶ Dobrando o contexto, o trabalho quadruplica e, na implementação padrão, a memória também. Já a feed-forward cresce de forma linear. Há duas respostas diferentes para esse problema.

Referências: Vaswani et al. (2017); Child et al. (2019).

C10-S02 · FlashAttention e atenção esparsa

30:05–30:40 · 00:35

Objetivo: apresentar duas famílias e o que cada uma muda, em visão geral.

NA TELA

Duas colunas. FlashAttention, uma matriz densa percorrida em blocos, com a memória rápida em destaque; rótulo "mesma atenção exata, menos acessos à memória". Atenção esparsa, com a matriz quase toda hachurada e poucos elementos calculados; rótulo "muda o padrão de atenção, $O(n\sqrt{n})$ em Child et al.".

CLIQUES

1. Janela de blocos varre a matriz densa.
2. Hachura cobre as células ignoradas da segunda matriz.

TEXTO FALADO

A primeira é o FlashAttention. Ele calcula a atenção exata, em blocos, para reduzir os acessos à memória da GPU. Não é uma aproximação e não elimina o custo quadrático das contas.

▶ A segunda são as abordagens esparsas, como o Sparse Transformer e o Longformer. Elas calculam só parte dos pares, o que reduz a complexidade, mas muda o padrão de atenção. Com essa base, por que essa arquitetura está em tantos lugares?

Referências: Dao et al. (2022); Child et al. (2019); Beltagy; Peters; Cohan (2020).

C11 · Famílias e aplicações

C11-S01 · BERT, GPT e T5

30:40–31:20 · 00:40

Objetivo: organizar as famílias de texto pela parte da planta que usam.

NA TELA

Três mini-plantas lado a lado, com a parte usada em tinta e o resto tracejado. Uma linha por modelo com tarefa de treino e ano.

CLIQUES

1. Cada mini-planta acende a parte usada.
2. Entra a linha de cada modelo.

> TEXTO FALADO
>
> As famílias de modelos de texto se organizam pela parte da planta que usam. O BERT, de 2018, usa só o encoder, com atenção bidirecional e masked LM. É bom para representar texto: busca, classificação e extração.
>
> ▶ O GPT usa só o decoder, com máscara causal. Do GPT-1 ao GPT-3, em 2020, com 175 bilhões de parâmetros, a estratégia foi escalar.
>
> ▶ O T5, de 2019, usa encoder e decoder e trata tudo como texto para texto: tradução, resumo e classificação, com o mesmo modelo e um prefixo na entrada. E a palavra token não precisa significar texto.

Referências: Devlin et al. (2018); Radford et al. (2018); Radford et al. (2019); Brown et al. (2020); Raffel et al. (2020).

C11-S02 · Visão, áudio e proteínas

31:20–32:05 · 00:45

Objetivo: mostrar a amplitude: o que vira token em cada domínio.

NA TELA

Três colunas com o mesmo formato "dado → tokens → modelo → saída",

- ViT: imagem em quadrados de 16×16 → encoder → classe;
- Whisper: espectrograma → encoder e decoder → texto;
- AlphaFold 2: sequência e alinhamentos → Evoformer e módulo de estrutura → coordenadas 3D. Ilustrações em traço técnico: grade de pixels, espectrograma em hachura, cadeia de pontos. Sem imagens de moléculas brilhantes.

CLIQUES

1. Em cada coluna, o dado é cortado em tokens.
2. Os tokens entram na caixa do modelo, e a saída é desenhada.

> TEXTO FALADO
>
> No ViT, a imagem é cortada em quadrados de 16 por 16 pixels. Cada quadrado é projetado linearmente e vira um token, com um encoder padrão e um token de classificação.
>
> ▶ No Whisper, de reconhecimento de fala, a entrada é um espectrograma log-Mel de 80 canais. Ele passa por duas camadas convolucionais e entra no encoder, e o decoder gera o texto.
>
> ▶ O AlphaFold 2 usa atenção em módulos próprios, como o Evoformer e a Invariant Point Attention. Não é um Transformer genérico aplicado a proteínas: o mérito inclui várias ideias específicas do domínio. Isso também vale para código e para várias mídias ao mesmo tempo.

Referências: Dosovitskiy et al. (2020); Radford et al. (2022); Jumper et al. (2021).

C11-S03 · Código, multimodais e leis de escala

32:05–32:45 · 00:40

Objetivo: fechar as aplicações e explicar a relação empírica entre recursos e desempenho.

NA TELA

Código de exemplo com a continuação sendo gerada (Codex). Imagem em traço e texto ligados a um mesmo espaço (CLIP, GPT-4). Gráfico log-log de perda por parâmetros: $L(N) \propto N^{-\alpha_N}$, com $\alpha_N \approx 0{,}076$.

CLIQUES

1. Linhas de código são completadas.
2. Imagem e texto são ligados por uma cota.
3. A reta do gráfico é desenhada; uma segunda anotação traz Chinchilla: dobrar o tamanho do modelo requer dobrar também os tokens de treino.

> **TEXTO FALADO**
>
> Modelos de código, como o Codex, são GPTs treinados em repositórios públicos.
>
> ▶ Modelos multimodais ligam imagem e texto: o CLIP faz isso num espaço comum, e o GPT-4 aceita imagens e texto como entrada.
>
> ▶ E as leis de escala: Kaplan e colegas mediram a perda caindo como uma lei de potência do número de parâmetros, com expoente de cerca de 0,076. Hoffmann e colegas refinaram: parâmetros e tokens de treino devem crescer juntos. São relações empíricas, medidas em certos intervalos, não leis da natureza. O que a escala não resolve?

Referências: Chen et al. (2021); Radford et al. (2021); OpenAI (2023); Kaplan et al. (2020); Hoffmann et al. (2022).

C12 · Limitações e questões em aberto

C12-S01 · Alucinação e contexto finito

32:45–33:20 · 00:35

Objetivo: explicar duas limitações ligadas ao próprio mecanismo.

NA TELA

Prancha com aparência de notas de revisão: anotações em vermelho sobre as plantas anteriores. Frase falsa e bem formada, escrita pelo grupo e rotulada "exemplo fabricado para ilustração", já que o mini-GPT não chega a produzir frases coerentes. Janela de contexto como régua, com tokens saindo pela borda.

CLIQUES

1. A nota vermelha "plausível não é verdadeiro" é desenhada.
2. A janela desliza e os tokens mais antigos somem.

> **TEXTO FALADO**
>
> *(Prancha vazia. Pausa de 2 s.)*
>
> ▶ Primeiro, a alucinação. Esta frase foi escrita por nós, só para ilustrar. O modelo é otimizado para produzir continuações prováveis, e nada nesse objetivo garante a verdade. Existem mitigações, mas é uma questão em aberto.
>
> ▶ Segundo, o contexto é finito. O que sai da janela, o modelo não vê. E, mesmo dentro dela, estudos mostram desempenho desigual conforme a posição da informação. Há também custos fora do modelo.

Referências: Ji et al. (2023); Liu et al. (2024).

C12-S02 · Custo energético e vieses

33:20–33:55 · 00:35

Objetivo: tratar impactos externos e encerrar as questões em aberto com rigor.

NA TELA

Tabela de materiais com a estimativa de Patterson et al. Para o treino do GPT-3, 1.287 MWh e 552 tCO_2e, com a fonte visível. Distribuição de próximo token após uma frase com profissão e gênero, rotulada "ilustração, não medição"; lista de questões em aberto.

CLIQUES

1. Linhas da tabela entram.
2. Barras da distribuição crescem, e a nota vermelha "o corpus entra no modelo" é desenhada.
3. Lista de questões em aberto entra.

> #### TEXTO FALADO
>
> Treinar modelos grandes consome energia. Patterson e colegas estimaram, para o treino do GPT-3, cerca de 1287 megawatts-hora e 552 toneladas de CO2 equivalente. E mostram que o local e o hardware importam muito.
>
> ▶ Há também os vieses: o modelo aprende as regularidades do corpus, incluindo estereótipos e sub-representação.
>
> ▶ Ficam questões em aberto: interpretabilidade, verificação, eficiência e avaliação de danos. Voltemos à prancha completa.

Referências: Strubell; Ganesh; McCallum (2019); Patterson et al. (2021); Bender et al. (2021).

C13 · Encerramento

C13-S01 · Síntese: o desenho de conjunto

`33:55–34:40 · 00:45`

Objetivo: reunir tudo em um único diagrama e voltar à tese.

NA TELA

A planta completa em tela cheia, com cada caixa numerada pela prancha em que foi explicada. Ao final, a frase-tese da abertura.

CLIQUES

1. Cada caixa da planta entra na sua posição, com o número da prancha em que foi explicada, uma por uma, seguindo a fala.
2. O fluxo de dados é desenhado da entrada à saída.
3. A frase-tese entra.

> #### TEXTO FALADO
>
> Esta é a planta completa, e cada caixa tem o número da prancha em que foi explicada. Os tokens viram embeddings, que recebem a posição. Depois vêm a atenção com várias cabeças, as residuais e a feed-forward, e a máscara no decoder.
>
> ▶ O treino é por próximo token ou por masked LM, e a geração é por amostragem.
>
> ▶ *(Pausa de 3 s com a frase na tela.)* A tese da abertura: a representação de cada token depende do contexto, e os pesos que dizem quanto cada posição aproveita das outras são aprendidos prevendo tokens. Tudo o que mostramos na tela tem código no repositório, com testes. Estas são as fontes.

Referências: Vaswani et al. (2017).

C13-S02 · Referências e créditos

`34:40–35:10 · 00:30`

Objetivo: dar visibilidade ao referencial teórico e aos integrantes.

NA TELA

Coluna esquerda, referências principais em ABNT, lidas de `10-referencias.md`. Coluna direita, nomes e papéis do config, licenças de fontes e o caminho do repositório.

CLIQUES

1. As referências entram em blocos por tema; a lista permanece parada tempo suficiente para ser lida em pausa.
2. Os nomes e papéis entram na coluna direita, seguidos de "Obrigado".

> **TEXTO FALADO**
>
> A base deste trabalho é o artigo de 2017, "Attention Is All You Need", e as fontes primárias de cada parte, com o material didático de Jurafsky e Martin e do Annotated Transformer.
>
> ▶ Em nome do grupo, Palos, Davi, Mendes e eu, Oliveira, obrigado pela atenção.

Referências: ver `10-referencias.md`.

Referências

Transformer e atenção

BAHDANAU, D.; CHO, K.; BENGIO, Y. Neural machine translation by jointly learning to align and translate. **arXiv**, 2014. Publicado em: International Conference on Learning Representations (ICLR), 2015. Disponível em: https://arxiv.org/abs/1409.0473.

DAO, T. et al. FlashAttention: fast and memory-efficient exact attention with IO-awareness. **arXiv**, 2022. Publicado em: Advances in Neural Information Processing Systems (NeurIPS), 2022. Disponível em: https://arxiv.org/abs/2205.14135.

VASWANI, A. et al. Attention is all you need. **arXiv**, 2017. Publicado em: Advances in Neural Information Processing Systems (NeurIPS), 2017. Disponível em: https://arxiv.org/abs/1706.03762.

Modelos sequenciais anteriores

BENGIO, Y.; SIMARD, P.; FRASCONI, P. Learning long-term dependencies with gradient descent is difficult. **IEEE Transactions on Neural Networks**, v. 5, n. 2, p. 157-166, 1994. DOI: 10.1109/72.279181.

CHO, K. et al. Learning phrase representations using RNN encoder-decoder for statistical machine translation. **arXiv**, 2014. Publicado em: Conference on Empirical Methods in Natural Language Processing (EMNLP), 2014. Disponível em: https://arxiv.org/abs/1406.1078.

GEHRING, J. et al. Convolutional sequence to sequence learning. **arXiv**, 2017. Publicado em: International Conference on Machine Learning (ICML), 2017. Disponível em: https://arxiv.org/abs/1705.03122.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. **Neural Computation**, v. 9, n. 8, p. 1735-1780, 1997. DOI: 10.1162/neco.1997.9.8.1735.

PASCANU, R.; MIKOLOV, T.; BENGIO, Y. On the difficulty of training recurrent neural networks. **arXiv**, 2012. Publicado em: International Conference on Machine Learning (ICML), 2013. Disponível em: https://arxiv.org/abs/1211.5063.

SUTSKEVER, I.; VINYALS, O.; LE, Q. V. Sequence to sequence learning with neural networks. **arXiv**, 2014. Publicado em: Advances in Neural Information Processing Systems (NeurIPS), 2014. Disponível em: https://arxiv.org/abs/1409.3215.

WILLIAMS, R. J.; ZIPSER, D. A learning algorithm for continually running fully recurrent neural networks. **Neural Computation**, v. 1, n. 2, p. 270-280, 1989. DOI: 10.1162/neco.1989.1.2.270.

Tokenização, normalização e posição

BA, J. L.; KIROS, J. R.; HINTON, G. E. Layer normalization. **arXiv**, 2016. Disponível em: https://arxiv.org/abs/1607.06450.

HE, K. et al. Deep residual learning for image recognition. In: IEEE CONFERENCE ON COMPUTER VISION AND PATTERN RECOGNITION (CVPR), 2016. **Proceedings** [...]. 2016. p. 770-778. DOI: 10.1109/CVPR.2016.90.

SCHUSTER, M.; NAKAJIMA, K. Japanese and Korean voice search. In: IEEE INTERNATIONAL CONFERENCE ON ACOUSTICS, SPEECH AND SIGNAL PROCESSING (ICASSP), 2012. **Proceedings** [...]. 2012. p. 5149-5152. DOI: 10.1109/ICASSP.2012.6289079.

SENNRICH, R.; HADDOW, B.; BIRCH, A. Neural machine translation of rare words with subword units. **arXiv**, 2015. Publicado em: Annual Meeting of the Association for Computational Linguistics (ACL), 2016. Disponível em: https://arxiv.org/abs/1508.07909.

SU, J. et al. RoFormer: enhanced transformer with rotary position embedding. **Neurocomputing**, v. 568, artigo 127063, 2024. DOI: 10.1016/j.neucom.2023.127063.

WU, Y. et al. Google's neural machine translation system: bridging the gap between human and machine translation. **arXiv**, 2016. Disponível em: https://arxiv.org/abs/1609.08144.

Famílias de modelos

BROWN, T. B. et al. Language models are few-shot learners. **arXiv**, 2020. Publicado em: Advances in Neural Information Processing Systems (NeurIPS), 2020. Disponível em: https://arxiv.org/abs/2005.14165.

CHEN, M. et al. Evaluating large language models trained on code. **arXiv**, 2021. Disponível em: https://arxiv.org/abs/2107.03374.

DEVLIN, J. et al. BERT: pre-training of deep bidirectional transformers for language understanding. **arXiv**, 2018. Publicado em: Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2019. Disponível em: https://arxiv.org/abs/1810.04805.

DOSOVITSKIY, A. et al. An image is worth 16x16 words: transformers for image recognition at scale. **arXiv**, 2020. Publicado em: International Conference on Learning Representations (ICLR), 2021. Disponível em: https://arxiv.org/abs/2010.11929.

JUMPER, J. et al. Highly accurate protein structure prediction with AlphaFold. **Nature**, v. 596, n. 7873, p. 583-589, 2021. DOI: 10.1038/s41586-021-03819-2.

OPENAI. GPT-4 technical report. **arXiv**, 2023. Disponível em: https://arxiv.org/abs/2303.08774.

RADFORD, A. et al. **Improving language understanding by generative pre-training**. OpenAI, 2018. Relatório técnico. Disponível em: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

RADFORD, A. et al. **Language models are unsupervised multitask learners**. OpenAI, 2019. Relatório técnico. Disponível em: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

RADFORD, A. et al. Learning transferable visual models from natural language supervision. **arXiv**, 2021. Publicado em: International Conference on Machine Learning (ICML), 2021. Disponível em: https://arxiv.org/abs/2103.00020.

RADFORD, A. et al. Robust speech recognition via large-scale weak supervision. **arXiv**, 2022. Publicado em: International Conference on Machine Learning (ICML), 2023. Disponível em: https://arxiv.org/abs/2212.04356.

RAFFEL, C. et al. Exploring the limits of transfer learning with a unified text-to-text transformer. **arXiv**, 2019. Publicado em: Journal of Machine Learning Research, 2020. Disponível em: https://arxiv.org/abs/1910.10683.

Escala, eficiência e amostragem

BELTAGY, I.; PETERS, M. E.; COHAN, A. Longformer: the long-document transformer. **arXiv**, 2020. Disponível em: https://arxiv.org/abs/2004.05150.

CHILD, R. et al. Generating long sequences with sparse transformers. **arXiv**, 2019. Disponível em: https://arxiv.org/abs/1904.10509.

FAN, A.; LEWIS, M.; DAUPHIN, Y. Hierarchical neural story generation. **arXiv**, 2018. Publicado em: Annual Meeting of the Association for Computational Linguistics (ACL), 2018. Disponível em: https://arxiv.org/abs/1805.04833.

HOFFMANN, J. et al. Training compute-optimal large language models. **arXiv**, 2022. Publicado em: Advances in Neural Information Processing Systems (NeurIPS), 2022. Disponível em: https://arxiv.org/abs/2203.15556.

HOLTZMAN, A. et al. The curious case of neural text degeneration. **arXiv**, 2019. Publicado em: International Conference on Learning Representations (ICLR), 2020. Disponível em: https://arxiv.org/abs/1904.09751.

KAPLAN, J. et al. Scaling laws for neural language models. **arXiv**, 2020. Disponível em: https://arxiv.org/abs/2001.08361.

POPE, R. et al. Efficiently scaling transformer inference. **arXiv**, 2022. Publicado em: Proceedings of Machine Learning and Systems (MLSys), 2023. Disponível em: https://arxiv.org/abs/2211.05102.

Limitações, interpretação e impacto

BENDER, E. M. et al. On the dangers of stochastic parrots: can language models be too big? In: ACM CONFERENCE ON FAIRNESS, ACCOUNTABILITY, AND TRANSPARENCY, 2021. **Proceedings** [...]. 2021. p. 610-623. DOI: 10.1145/3442188.3445922.

CLARK, K. et al. What does BERT look at? An analysis of BERT's attention. **arXiv**, 2019. Disponível em: https://arxiv.org/abs/1906.04341.

JAIN, S.; WALLACE, B. C. Attention is not explanation. **arXiv**, 2019. Publicado em: Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), 2019. Disponível em: https://arxiv.org/abs/1902.10186.

JI, Z. et al. Survey of hallucination in natural language generation. **ACM Computing Surveys**, v. 55, n. 12, p. 1-38, 2023. DOI: 10.1145/3571730.

LIU, N. F. et al. Lost in the middle: how language models use long contexts. **Transactions of the Association for Computational Linguistics**, v. 12, p. 157-173, 2024. DOI: 10.1162/tacl_a_00638.

PATTERSON, D. et al. Carbon emissions and large neural network training. **arXiv**, 2021. Disponível em: https://arxiv.org/abs/2104.10350.

STRUBELL, E.; GANESH, A.; MCCALLUM, A. Energy and policy considerations for deep learning in NLP. **arXiv**, 2019. Publicado em: Annual Meeting of the Association for Computational Linguistics (ACL), 2019. Disponível em: https://arxiv.org/abs/1906.02243.

Material didático

JURAFSKY, D.; MARTIN, J. H. **Speech and language processing**. 3. ed. Rascunho, versão de 19 ago. 2026. Disponível em: https://web.stanford.edu/~jurafsky/slp3/.

RUSH, A. The annotated transformer. In: WORKSHOP FOR NLP OPEN SOURCE SOFTWARE (NLP-OSS), 2018. **Proceedings** [...]. 2018. p. 52-60. DOI: 10.18653/v1/W18-2509. Versão atualizada disponível em: https://nlp.seas.harvard.edu/annotated-transformer/.

Dados e recursos usados nas demos

ASSIS, M. de. **Memorias posthumas de Braz Cubas**. Project Gutenberg, ebook 54829. Domínio público. Disponível em: https://www.gutenberg.org/ebooks/54829.

ASSIS, M. de. **Quincas Borba**. Project Gutenberg, ebook 55682. Domínio público. Disponível em: https://www.gutenberg.org/ebooks/55682.

Fontes: Barlow, Barlow Condensed e IBM Plex Mono (SIL Open Font License 1.1), licenças em `assets/fonts/`. KaTeX 0.16.11 (licença MIT), em `assets/vendor/katex/`.